

Open in app ↗

Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Towards AI · Following

Prompt Engineering Is Dead for AI Agents — Here Is What Actually Works — Context Engineering

8 min read · May 3, 2026



Darshandagaa

[Follow](#)

Listen



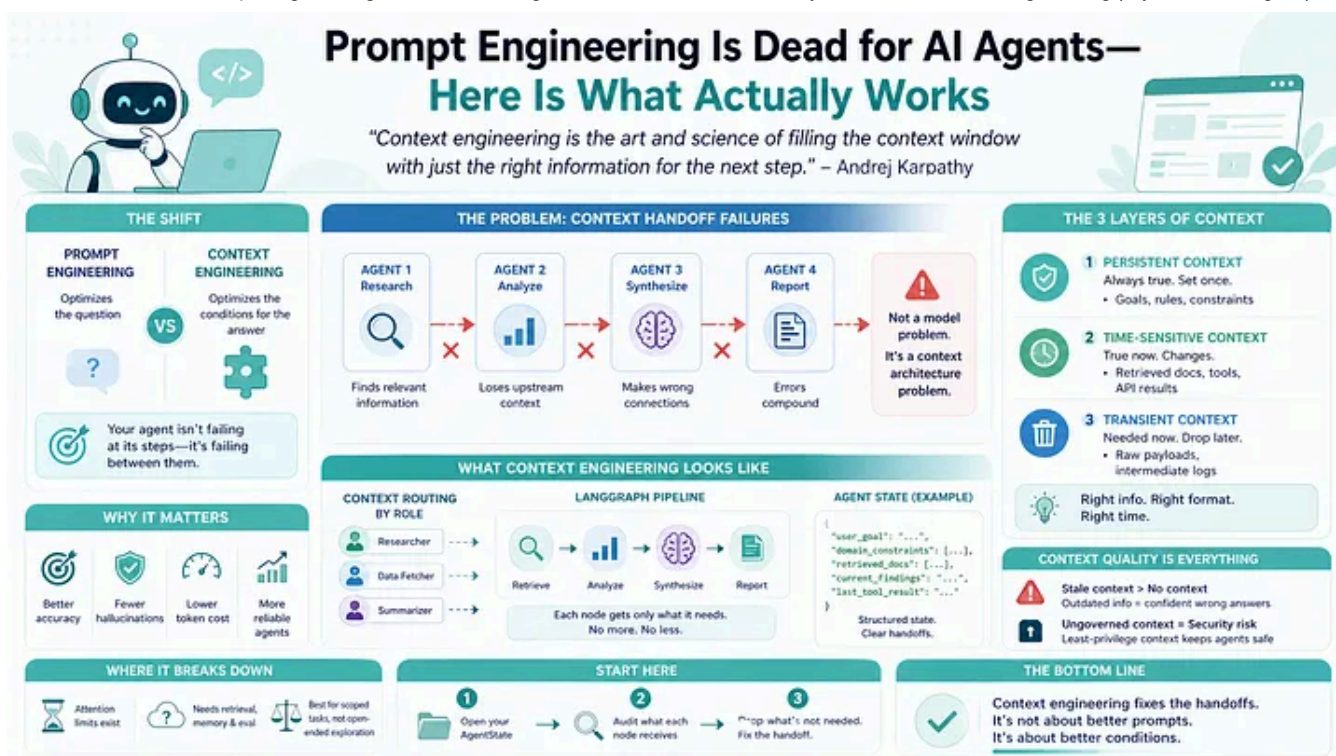
Share



More

“Context engineering is the delicate art and science of filling the context window with just the right information for the next step.” — Andrej Karpathy, former Tesla AI Director, OpenAI co-founder.

My first reaction when I read that? Relief. Someone had finally named the thing I had been fighting in production for an year without a word for it.



1.1 Image generated by GPT

I have spent the better part of three years building multi-agent systems in LangGraph — financial research agents, healthcare pattern discovery pipelines, NL-to-SQL orchestration layers for enterprise clients. Every single one of those systems had the same failure profile. The prompts were fine. The models were capable. The logic looked clean in a Jupyter notebook.

Then we went to production and things fell apart — not at the individual steps, but *between* them.

A healthcare summarisation agent would pull the right documents. Its successor in the graph would receive a stripped-down state object and proceed to hallucinate patient history that had been sitting three nodes upstream. The model had not failed. The context handoff had. I spent weeks tuning system prompts trying to fix something that was never a prompt problem.

That is the gap context engineering is designed to close. And in 2026, it is the discipline separating agents that work from agents that demo.

TL;DR: Prompt engineering optimises the question. Context engineering optimises the conditions under which the question is answered. Your agent is not failing at its

steps — it is failing between them because the wrong information is present at each node. Context engineering is how you fix that.

The Problem Nobody Names Until It Bites Them

Here is how most teams build their first multi-agent system. They spend serious time on the system prompt. They write clear task descriptions, add output format instructions, throw in a few examples. The individual agent responds well in isolation.

Then they connect three agents together and the whole thing degrades.

The second agent does not know what the first one decided. The third agent invents context the second never produced. By step four, the agent is confident and completely wrong — not because it reasoned poorly, but because it had no idea what had actually happened upstream.

Think of it like handing a relay baton — but nobody tells the next runner where the race has been or which lane to stay in. The runner is capable. The problem is the handoff.

This is not a model capability issue. Research by Chroma published in July 2025, which tested 18 LLMs including Claude 4, GPT-4.1, and Gemini 2.5, found something stark: model performance degrades non-uniformly as context grows, even on trivially simple retrieval tasks [1]. The models are not stupid. Attention is not free. What goes into the context window — and what gets left out — determines almost everything about what comes out.

Prompt engineering never solved this. It was never designed to. A well-crafted system prompt tells the model what to do. Context engineering tells the model what it needs to know *right now* to do it well.

What Context Engineering Actually Is

The clearest definition I have found: context engineering is the discipline of designing dynamic systems that provide the right information, in the right format,

at the right time — so the model has everything it needs to complete the next step [2].

It is infrastructure work, not linguistic work. You are not writing better sentences. You are architecting what flows through your agent graph at every transition.

When I refactored my LangGraph-based financial research agent, the change was not in any prompt. It was in the `AgentState TypedDict`. I stopped treating state as a scratchpad and started treating it as a contract. Every node declared what it needed to read and what it was responsible for writing. The downstream agents stopped hallucinating because they were no longer guessing what had happened upstream — it was in the state, structured and retrievable.

The three layers that matter in production are these:

Persistent context is what the agent should always know: the user's goal, their constraints, prior session decisions, domain-specific rules. In my healthcare pipelines, this meant patient context, regulatory constraints, and risk thresholds — injected once at graph initialisation and never dropped.

Time-sensitive context is what is true right now but will change: retrieved documents, current tool outputs, API responses, the result of the last reasoning step. This is where RAG fits — not as the whole solution, but as one layer of a larger context architecture. The retrieval is easy. The hard part is deciding which retrieved content gets passed forward, at what level of compression, and in what format.

Transient context is what the current step needs but the next one does not: raw API payloads, verbose intermediate reasoning, tool call logs. This is the context you should be actively dropping, not hoarding. Every unnecessary token in the context window is competing for attention with the tokens that matter.

Most production failures I have debugged involve one of three mistakes: persistent context that was never injected, time-sensitive context that went stale between steps, or transient context that was never cleaned up and polluted later reasoning.

What This Looks Like in a LangGraph Pipeline

Here is a stripped-down version of how I structure state for a multi-agent research pipeline. The point is not the code — it is the discipline it reflects.

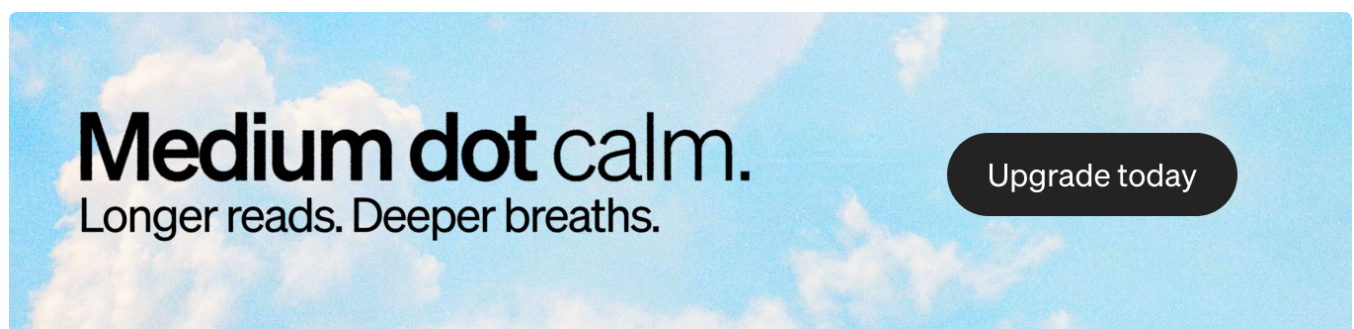
```
from typing import TypedDict, Optional, List
from langgraph.graph import StateGraph

class ResearchAgentState(TypedDict):
    # Persistent context — set once, never overwritten
    user_goal: str
    domain_constraints: List[str]
    session_id: str

    # Time-sensitive context — updated by retrieval/tool nodes
    retrieved_docs: List[str]
    current_findings: str
    last_tool_result: Optional[str]

    # Transient context — cleared after use
    raw_api_payload: Optional[str] # cleared after parsing
    intermediate_reasoning: Optional[str] # cleared after synthesis
```

Every node in the graph receives this state and is responsible for updating only its declared slice. The orchestrator node at the top decides which sub-agents fire and what subset of state they receive.



This is context routing. A coding sub-agent does not need the retrieved medical literature from the domain specialist. A summarisation agent does not need the raw API payload from the data fetcher. Sending every agent everything is the equivalent of briefing your entire company before every meeting — technically complete, operationally catastrophic.

In my production systems, routing context by role reduced token usage measurably and, more importantly, reduced hallucination in specialist agents. They were no

longer distracted by irrelevant information competing for attention in a finite context window.

MCP (Model Context Protocol) is accelerating this pattern at scale. By late 2025, there were over 10,000 public MCP servers deployed — a standardised interface that lets agents query tools, databases, and external systems in a structured way [3]. What MCP does architecturally is externalise context sources so agents can pull what they need rather than receiving everything pre-loaded. It is the difference between a library and a briefing document. The agent goes to the shelf when it needs a specific book. It does not carry the library with it.

The Part Nobody Talks About: Context Quality

A 2026 survey found that 82% of IT and data leaders now agree that prompt engineering alone is no longer sufficient for production AI [4]. The budget is following: 95% of data teams plan to invest in context engineering capability in 2026.

But most of the discourse focuses on context *volume* — bigger context windows, more retrieved chunks, more tools available. The actual bottleneck in my experience is context *quality*.

Stale context is worse than no context. An agent that confidently acts on yesterday's retrieved document, unaware it has been superseded, produces errors that look like good reasoning. In my NL-to-SQL pipelines for enterprise clients, schema drift was the single most consistent failure mode. The SQL generation was correct — for a schema that no longer existed. The prompt was fine. The context was wrong.

Ungoverned context is a security problem. When agents have access to everything in the state — including data from other users, prior sessions, sensitive tool outputs — you are one prompt injection away from a context leak. Least-privilege context design is not a nice-to-have. With 86% of CISOs reporting no enforcement policy for AI agent access [5], context governance is where the next wave of enterprise AI incidents will originate.

Where This Breaks Down

Context engineering does not solve the fundamental attention limitation. The Chroma study showed that even with well-structured context, model reliability degrades as input length increases [1]. Engineering better context buys you more reliable behaviour — it does not give you infinite reliable context.

For tasks where the context is genuinely massive — long-running autonomous agents operating over hours, research synthesis across hundreds of documents — context engineering must be paired with aggressive summarisation, hierarchical retrieval, and explicit memory management. Context engineering tells you *what* to put in the window. Memory architecture tells you what to do when the window is not enough.

Context engineering also does not substitute for evaluation. A beautifully structured context pipeline that consistently retrieves the wrong documents will fail with confidence. The retrieval layer, the memory layer, and the routing logic all need their own evaluation passes — separate from model output quality.

This works when your agents operate over well-scoped tasks with stable context requirements. It does not work well for open-ended, exploratory agents where the relevant context cannot be determined in advance. For those cases, dynamic retrieval with explicit uncertainty signalling to the model is the better fit.

Start Here, Not at Your System Prompt

If you are building a LangGraph agent today, open your `AgentState` definition. Ask three questions: What does each node assume is already in state? What does each node actually receive? What gets dropped before it is needed?

That gap — between what nodes assume and what they receive — is where your agent is failing right now. It is not your system prompt.

If you are earlier in the stack, start by reading Andrej Karpathy's thread on context engineering and the Chroma Research context length study [1]. Both are short and will reframe how you think about every agent you build next.

The most impactful upgrade you can make to an existing multi-agent system is not switching to a newer model. It is auditing what is in your context window at every step, dropping what does not belong, and ensuring what does belong is current, structured, and scoped to the agent receiving it.

Start with one node. Audit its inputs. That is context engineering. The reliability improvement starts there — and it compounds quickly.

References

- [1] Chroma Research, “LLM Context Length Study: Testing Models on Retrieval Reliability,” July 2025. <https://research.trychroma.com/context-length-study>
- [2] Firecrawl, “Context Engineering vs Prompt Engineering for AI Agents,” February 2026. <https://www.firecrawl.dev/blog/context-engineering>
- [3] Salesforce, “8 Ways AI Agents Are Evolving in 2026,” May 2026. <https://www.salesforce.com/blog/ai-agent-trends-2026/>
- [4] DataHub, “Context Engineering vs Prompt Engineering: 2026 State of Context Management Report,” April 2026. <https://datahub.com/blog/context-engineering-vs-prompt-engineering/>
- [5] AI Agent Store, “Daily AI Agent News — Last 7 Days,” May 2026. <https://aiagentstore.ai/ai-agent-news/this-week>

[Prompt Engineering](#)[AI Agent](#)[Python](#)[Data Science](#)[AI](#)[Following](#)

Published in Towards AI

138K followers · Last published 2 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Follow

Written by Darshandagaa

473 followers · 31 following

I like building, breaking and rebuilding stuff!

